

# INSTITUCIÓN UNIVERSITARIA DE COLOMBIA

## SESIÓN 10 SVM – ALGORITMO DE SUPPORT VECTOR MACHINE

### I. DATOS GENERALES DE LA ASIGNATURA

|                           |                                |
|---------------------------|--------------------------------|
| <b>Asignatura:</b>        | Inteligencia Artificial II     |
| <b>Período Académico:</b> | IV Semestre                    |
| <b>Créditos:</b>          | 3 créditos                     |
| <b>Duración en Horas:</b> | 40 horas                       |
| <b>Docente:</b>           | Amaury Giovanni Méndez Aguirre |

### CONTEXTO: LA LÍNEA PERFECTA

A diferencia de KNN (que requiere memorizar todo), hay algoritmos que aprenden a dibujar una frontera matemática. Las **Máquinas de Vectores de Soporte (SVM - Support Vector Machines)** son elegantes y matemáticamente rigurosas. Su objetivo no es solo separar los gatos de los perros (o las transacciones legítimas de los fraudes), sino trazar una línea que tenga el *mayor margen de seguridad (calle)* posible entre las dos clases.

Antes del boom del Deep Learning en 2012, SVM era el rey indiscutible de la Inteligencia Artificial. Aún hoy, para datasets pequeños y problemas de alta dimensión, suele superar a las Redes Neuronales.

## 1. Hiperplanos y Vectores de Soporte

En 2D, la frontera es una línea. En 3D, es un plano. En N-dimensiones, la llamamos **Hiperplano**. El algoritmo de SVM busca maximizar el "Margen", es decir, la distancia perpendicular entre el hiperplano y los puntos de datos más cercanos de cada clase. Curiosamente, a la IA no le importan los miles de puntos que están lejos de la frontera; solo le importan los puntos críticos que están al borde del margen. Esos puntos solitarios sostienen la estructura matemática y se denominan **Vectores de Soporte**.

**El Truco del Kernel (Kernel Trick):** ¿Qué pasa si los datos forman un anillo concéntrico y es imposible trazar una línea recta que los separe? SVM usa "Kernels" (como el RBF o polinomial) para proyectar matemáticamente los datos a una dimensión superior (como un embudo 3D) donde un plano recto sí pueda cortarlos perfectamente.

### TALLER ANALÍTICO: DIBUJANDO EL MARGEN (20 MIN)

Dibujen en su cuaderno un plano cartesiano (ejes X e Y de 0 a 10). Ubiquen los siguientes puntos:

- Clase A (Círculos): (2,2), (3,3), (4,2).
- Clase B (Equis): (6,6), (7,8), (8,7).

1. Tracen con regla la línea recta óptima que separe ambos grupos buscando dejar el mismo espacio de "calle" a ambos lados.
2. Marquen con un círculo rojo cuáles de esos 6 puntos actuarían como los "Vectores de Soporte" de su modelo.
3. Si agregamos un nuevo punto de Clase A en la coordenada (1,1), ¿cambiaría la posición de la línea? Justifiquen por qué sí o por qué no basándose en la teoría.

*Espacio para análisis del margen...*

## 2. Implementación de SVM en Python

La implementación en Scikit-Learn es directa. El parámetro crucial aquí es el tipo de `kernel`. Un kernel `'linear'` trazará una recta rígida, mientras que un kernel `'rbf'` (Radial Basis Function) envolverá los puntos creando fronteras curvas altamente complejas.

```
import numpy as np
from sklearn.svm import SVC

# 1. Crear el dataset (X = Coordenadas, Y = Etiquetas binarias 0 o 1)
X = np.array([ [2,2], [3,3], [4,2], [6,6], [7,8], [8,7] ])
Y = np.array([ 0, 0, 0, 1, 1, 1 ])

# 2. Inicializar SVM con Kernel Lineal
modelo_svm = SVC(kernel='linear')

# 3. Entrenar el modelo (Aprender la ecuación del hiperplano)
modelo_svm.fit(X, Y)

# 4. Extraer los Vectores de Soporte descubiertos por la IA
vectores = modelo_svm.support_vectors_
print("Los Vectores de Soporte son:\n", vectores)

# 5. Predicción
nuevo_punto = np.array([5, 4])
pred = modelo_svm.predict(nuevo_punto)
print("El punto [5,4] pertenece a la clase:", pred[0])
```

## TALLER DE LABORATORIO: FRONTERAS NO LINEALES (40 MIN)

### Misión Práctica:

1. Transcriban y ejecuten el código superior. Observen los vectores de soporte impresos. ¿Coinciden con los que marcaron en su cuaderno durante el taller analítico?
2. Ahora, vamos a "engañar" a la frontera lineal. Agreguen un nuevo punto a la variable X: [5, 5] y asígñenle la etiqueta 0 (Clase A) en la variable Y.
3. Vuelvan a entrenar el modelo lineal. Intentará trazar una recta pero probablemente fallará o se verá forzado.
4. Cambien el hiperparámetro de `kernel='linear'` a `kernel='rbf'`. Vuelvan a entrenar y a predecir.
5. **Reflexión:** El "Kernel Trick" (RBF) le permite al SVM aislar puntos que están "rodeados" por la clase enemiga. ¿En qué escenario del mundo real (ej. medicina o reconocimiento facial) creen que un kernel lineal fallaría completamente y se requeriría RBF?

*Material preparado por: Docente Amaury Giovanni Méndez Aguirre — 2025  
Institución Universitaria de Colombia — Ingeniería de Sistemas — VIII Semestre*